

FPGA-Based Dynamically Reconfigurable SQL Query Processing

DANIEL ZIENER, FLORIAN BAUER, ANDREAS BECHER, CHRISTOPHER DENNL,
and KLAUS MEYER-WEGENER, Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU)
UTE SCHÜRFELD, IBM Deutschland Research & Development GmbH
JÜRGEN TEICH, Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU)
JÖRG-STEPHAN VOGT and HELMUT WEBER, IBM Deutschland Research
& Development GmbH

In this article, we propose an FPGA-based SQL query processing approach exploiting the capabilities of partial dynamic reconfiguration of modern FPGAs. After the analysis of an incoming query, a query-specific hardware processing unit is generated on the fly and loaded on the FPGA for immediate query execution. For each query, a specialized hardware accelerator pipeline is composed and configured on the FPGA from a set of presynthesized hardware modules. These partially reconfigurable hardware modules are gathered in a library covering all major SQL operations like restrictions and aggregations, as well as more complex operations such as joins and sorts. Moreover, this holistic query processing approach in hardware supports different data processing strategies including row- as column-wise data processing in order to optimize data communication and processing. This article gives an overview of the proposed query processing methodology and the corresponding library of modules. Additionally, a performance analysis is introduced that is able to estimate the processing time of a query for different processing strategies and different communication and processing architecture configurations. With the help of this performance analysis, architectural bottlenecks may be exposed and future optimized architectures, besides the two prototypes presented here, may be determined.

Categories and Subject Descriptors: B.5.1 [Register-Transfer-Level Implementation]: Design—*Data-path design*; C.3 [Special-Purpose and Application-based Systems]: Microprocessor/Microcomputer Applications; H.2.4 [Database Management]: Systems—*Query processing*

General Terms: Design, Performance

Additional Key Words and Phrases: FPGA, dynamic partial reconfiguration, SQL processing, reconfigurable computing

ACM Reference Format:

Daniel Ziener, Florian Bauer, Andreas Becher, Christopher Dennl, Klaus Meyer-Wegener, Ute Schürfeld, Jürgen Teich, Jörg-Stephan Vogt, and Helmut Weber. 2016. FPGA-based dynamically reconfigurable SQL query processing. *ACM Trans. Reconfigurable Technol. Syst.* 9, 4, Article 25 (August 2016), 24 pages. DOI: <http://dx.doi.org/10.1145/2845087>

Authors' addresses: D. Ziener, F. Bauer, A. Becher, C. Dennl, and J. Teich, Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU) Cauerstraße 11, 91058 Erlangen, Germany; emails: {daniel.ziener, florian.bauer, andreas.becher, christopher.dennl}@fau.de, teich@cs.fau.de; K. Meyer-Wegener, Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU) Martensstraße 3, 91058 Erlangen, Germany; email: klaus.meyer-wegener@fau.de; U. Schürfeld, J.-S. Vogt, and H. Weber, IBM Deutschland Research & Development GmbH Schönaicher Str. 220, 71032 Böblingen, Germany; emails: {uteschuerfeld, JSVOGT}@de.ibm.com.

Author's current address: D. Ziener, Hamburg University of Technology (TUHH), Am Schwarzenberg-Campus 3 (E), 21073 Hamburg, Germany; email: daniel.ziener@tuhh.de.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 2 Penn Plaza, Suite 701, New York, NY 10121-0701 USA, fax +1 (212) 869-0481, or permissions@acm.org.

© 2016 ACM 1936-7406/2016/08-ART25 \$15.00

DOI: <http://dx.doi.org/10.1145/2845087>

1. INTRODUCTION

One of the big challenges in the IT sector today is the handling of an ever-growing amount of data. The annual amount of data that is created, captured, or replicated worldwide increases by a factor of 10 every 5 years [Gantz et al. 2008]. Storing all these data is one challenge; processing and analyzing them on demand is another one. In fact, the growth of data even outnumbers Moore's law of digital-circuit complexity. This means that it is impossible to handle this amount of data by just using more processing resources. At the same time, the energy consumption of all data centers worldwide increases by 16.7% each year [Koomey 2008]. In 2013, the energy consumption of all IT facilities together approached 10% of the worldwide energy generation [Mills 2013], and it will increase further. This forces us to investigate more efficient data-processing approaches.

The more important kinds of data are stored in *relational database management systems* (RDBMSs), which are able to load, analyze, and process the data based on requested inquiries, mostly given in the form of *SQL queries* [SQL 2008]. Therefore, efficient query processing can on the one hand increase the performance of a database processing system and on the other hand reduce the energy consumption. Hardware accelerators are able to reach the goal of increasing the efficiency by exploiting natural parallelism of reconfigurable hardware, as available in FPGAs. Consequently, it is worthwhile to investigate the use of FPGAs in the field of database applications in general and for query execution purposes in particular. In analyzing the data stored in huge tables, which are common in *data warehouse applications* [Inmon 2002], the offload of *restrictions*, *joins*, and *aggregations* to an FPGA can increase the efficiency of query processing substantially.

However, a static hardware approach is not suitable to cover all operator types used in these queries. With the help of *partial dynamic reconfiguration*, however, techniques for query-specific data-path generation can be used to increase the efficiency of FPGA-based accelerators. Partially reconfigurable modules stemming from a library of SQL operators can be composed on the fly to form a query data path and can be loaded onto the FPGA. Moreover, this library can be extended continuously to offer new operators or SQL constructs.

In this article, we give an overview of such a novel FPGA-based partially reconfigurable query processing system. This system consists of a static hardware part, including communication and configuration interfaces; a library of partially reconfigurable modules, which covers all common SQL operators; and software running on the host system to analyze an incoming query, select partially reconfigurable modules from the library, and determine feasible placements for them, as well as controlling the communication to and configuration of the FPGA. This query processing system has evolved from a first publication of basic concepts [Dennl et al. 2012] to a mature and complete design flow for query processing as presented in this article. Starting with just restrictions [Dennl et al. 2012], aggregations [Dennl et al. 2013], and join and sort [Becher et al. 2014], the novel hash-join and column-based processing modules, introduced in this article for the first time, complete the portfolio of supported operations. One of the main challenges was the combination of streaming-based operations with operations using multiple inputs and buffers for storing intermediate results. In our proposed system architecture, this is solved by introducing different kinds of partial reconfigurable areas for SQL operators implemented in a streamed-based manner and modules using more than one input and/or external memory access. Furthermore, investigations to process table data column-wise by introducing special modules in order to reduce the amount of incoming data are analyzed and presented. The main idea is to use the given static system and only introduce new modules that are able to process data column-wise and modules doing row-to-column transformations. Moreover,

a calculus for performance assessment is presented in this article that gives us the possibility to evaluate the processing time of a query on different, even nonexistent architectures with different parameters, for example, throughput of communication interfaces. It also allows us to explore the rich facets of different processing possibilities by combining different modules and different implementations of the same query operations, for example, processing a join operation as a hash or rather a sort-merge-join. With the help of this performance calculus, we are able to choose at runtime the best configuration of our SQL processing system for a given incoming query, database size, and estimated selectivity of the query. This is a prerequisite to later enhance the query optimizer with cost models that take our processing system into account.

The remainder of the article is outlined as follows: Section 2 discusses related work in the field. Section 3 introduces the concept of the proposed SQL processing approach and gives an overview of the methodology. The integration in an existing *database management system* (DBMS) is described in Section 4. The implemented module library including all common SQL operations is introduced in Section 5, and the performance calculus is presented in Section 6. A comparison between row- and column-wise processing is the subject of Section 7, and the developed prototypes are introduced in Section 8. Experimental results that compare our prototypes with software-only approaches and a comparison of the processing time of a query under different system configurations are presented in Section 9. Finally, a conclusion is given in Section 10.

2. RELATED WORK

In this section, existing work on hardware acceleration of SQL query processing is summarized. *Netezza FAST engines* [IBM 2014] is a commercially available platform that uses FPGA support for query acceleration. It provides a static system with a limited set of parameters, which only covered projection and restriction operators up to now. Enhancing such a static system to support more operators leads to high area requirements, regardless of whether the operators are used or not. Another approach is provided by *Glacier* [Mueller et al. 2010], an SQL-to-VHDL compiler, which can compile SQL queries into synthesizable VHDL. Other approaches accelerate specific database operation, like hash-joins [Halstead et al. 2013] or the skyline operator [Woods et al. 2013]. Furthermore, FPGA-based XML filters [Teubner et al. 2013] might also help to offload operations from a processor to an FPGA. Takenaka et al. [2012] showed how to use FPGAs for SQL-based event processing. However, the need for a complete synthesis makes all these approaches useful only for environments with a fixed and known set of queries. Static hardware designs are inappropriate for two reasons: either they support only a small set of query operators or the on-the-fly synthesis of query-specific designs takes far too long.

Configurable engines, as in IBM's *Netezza* approach, might be a suitable compromise. One example is the *Ibex* storage engine, which filters the incoming data with restriction and aggregation operators implemented on an FPGA [Woods et al. 2014]. This filtering engine is placed between the disks and the processor and efficiently reduces the data volume that has to be processed by the processor. Another example is the system proposed in Sukhwani et al. [2015], which covers restrictions and data sorting also with the help of configurable engines. The reconfiguration of such engines for processing multiple queries might be faster compared to our partial reconfiguration approach. However, it restricts the flexibility of designing optimized modules for new operators and, therefore, limits the extendability and efficiency of the system.

Werner et al. [2013] analyzed different hardware implementations of joins used in semantic web databases. They found the implementation of the merge-join to be the fastest join variant. The analysis was done with stand-alone modules only. However, the goal here is to provide join modules for a partially reconfigurable accelerator system.

Finally, design flows supporting the design of partially reconfigurable systems are summarized: Xilinx integrated the partial reconfiguration feature in their design tool *PlanAhead* [Xilinx 2011]. Also, Altera supports partial reconfiguration for the new *Stratix V* series and integrates a partial design flow in their tools that is quite similar to the Xilinx approach [Bourgeault 2011]. However, all these approaches support only an *island reconfiguration style*, which means that a partially reconfigurable module can only be placed in one dedicated area and this area can only be occupied by a single module at the same time. To overcome these restrictions, the FPGA research community has introduced some partial design flows that are able to place several modules into a partially reconfigurable area. Some of them allow even module relocation. The approach proposed in the following also relies on one of such tools named *GoAhead* [Beckhoff et al. 2012], which provides the generation of communication structures for data-flow-oriented communications, as well as on the partial flow from Xilinx *PlanAhead*.

3. CONCEPT AND SYSTEM OVERVIEW

In this section, we describe our concept of processing SQL operations using partial dynamic reconfiguration. First, an overview of our system architecture is presented. Next, a closer look at the basic architecture inside the FPGA is given.

The major concept of on-the-fly query acceleration through partial dynamic FPGA reconfiguration is depicted in Figure 1. An incoming query (see Listing 1) is analyzed by an *optimizer* and the corresponding *query execution plan* is generated and processed by a component called *Reconfiguration Manager* running on the host. Operator modules from the module library, which reside in the main memory of the host, are selected and the query data path is composed on the fly. After that, the streaming data path, which typically cascades several modules, is loaded into a partially reconfigurable area inside the FPGA. Furthermore, the reconfiguration manager keeps track of the allocation of partially reconfigurable areas as well as active and finished queries.

After module loading, the database tables are streamed from the main memory to the FPGA and into the partially reconfigurable area. Hereby, the data is processed by the loaded operator modules and the corresponding result is streamed continuously back to the main memory. Each partially reconfigurable area may implement one or a subset of a query accelerator, and each may consist of one or more partially reconfigurable modules. Figure 1 shows also the partitioning of partially reconfigurable areas into elementary units called *slots* forming a grid for the placement of modules. Each module occupies one or more neighboring slots. Several concatenated modules finally implement a specific query accelerator. The communication inside a partial reconfigurable area between different modules is based on the I/O bar concept. An I/O bar is a composition of several prerouted wires forming a data bus whose purpose is to pipe a data stream. Partial modules can be plugged on and off the I/O bar at runtime and have the possibility to bypass or manipulate the data stream. Such communication structures can be built by using the *GoAhead* tool [Beckhoff et al. 2012] (see also Dennl et al. [2012] for more details).

Inside the FPGA, *tuples* are processed in a fully pipelined manner by a chain of modules. Usually, the bit width of a tuple is wider than the width of a communication data bus and may even add up to several kilobytes. Thus, it is impossible to process whole tuples at the speed of just one clock cycle. Therefore, tuples are divided into smaller entities called *chunks* that match the data bus width and contain one or several *attributes*. To achieve the best possible throughput, each module has been developed to process one chunk per clock cycle. It may still have a latency of multiple clock cycles.

Chunks must be properly sequenced (see Figure 2). Modules access exactly the attributes needed for their operation by fetching the appropriate chunks from the data

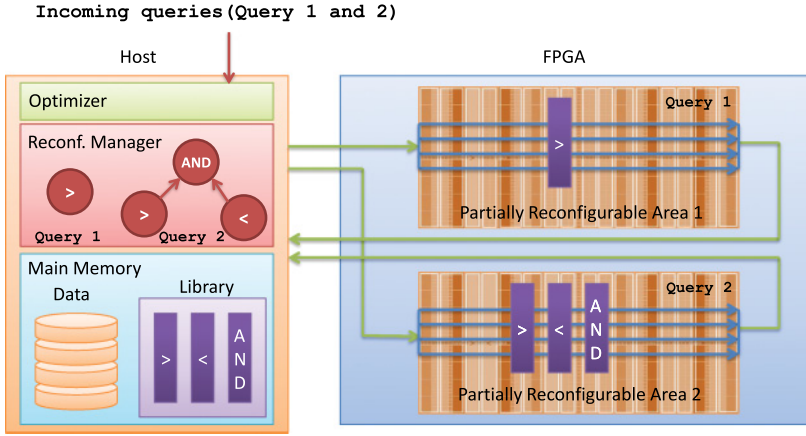


Fig. 1. An overview of the proposed query acceleration system: each incoming query is analyzed by the Reconfiguration Manager and corresponding modules of the generated query execution plan are loaded subsequently onto the FPGA. On the right, two partially reconfigurable areas are depicted that even allow the simultaneous processing of two (partial) queries (Queries 1 and 2) in parallel.

SELECT * FROM table WHERE age > 20	Query 1
SELECT * FROM table WHERE salary > 10000 AND year < 1990	Query 2

Listing 1. Two incoming example queries.

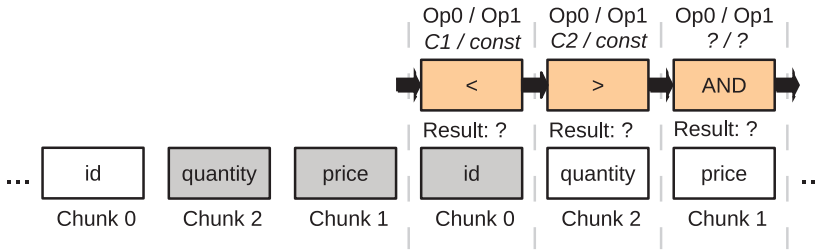


Fig. 2. Snapshot of processing tuples. In the example, tuples consist of three attributes and three chunks, and the modules are configured with indices that denote the position of the operands. The shown data path consists of three cascaded modules. It is not determined yet how the results are handled. Figure 3 shows the completed snapshot.

stream. Each chunk has a unique index and the modules are configured with the right chunk indices; for example, if an adder module needs chunks 2 and 4 as inputs, it is configured with index 2 for the first operand and with index 4 for the second. Each module counts the bypassing chunks and loads its operand registers when the right chunk arrives.

The modules are also configured with the number of chunks per tuple. Thus, a module recognizes when a tuple ends and a new tuple begins. This information is important because projections and restrictions are evaluated row-wise; that is, each tuple is evaluated independently. Operands not only are attributes but also can be configured as constant values. As stated earlier, attributes must be included in single chunks in order to address them via indices. Figure 2 shows a snapshot of a typical data-processing stream. The modules are configured with the appropriate chunk indices, which denote the position of the operands inside the tuple.

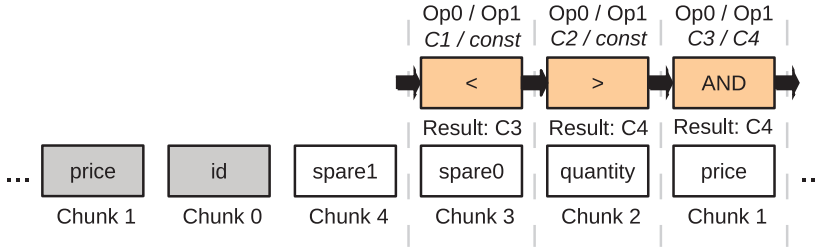


Fig. 3. Adjusting the data processing introduced in Figure 2 to handle intermediate results. *Spare chunks* are appended to the end of each tuple to provide space for intermediate results.

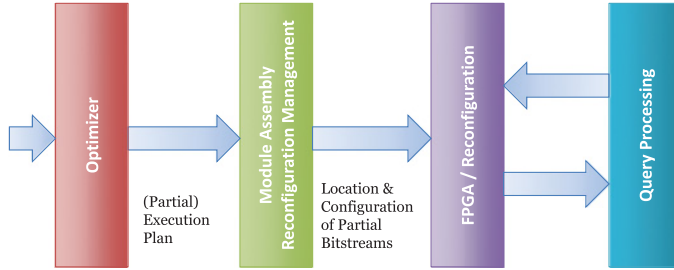


Fig. 4. The query will be analyzed by an optimizer that generates a (partial) execution plan. The corresponding modules are selected, configured, and placed by the reconfiguration manager. Finally, the modules are loaded onto the FPGA and the query is beginning to be processed.

To include intermediate results in the tuple stream, a sufficient number of *spare chunks* per tuple is allocated in advance. The modules are then configured with the proper result index. Hence, each module knows where it has to insert its results. Figure 3 shows an adjusted tuple-stream processing. Modules are configured with an additional result index.

The minimum number of spare chunks, the assignment to intermediate results, and the placement order of operator modules inside the partially reconfigurable area from left to right can be determined by using query expression trees with posttraversal linearization. A more detailed description of these tasks can be found in Dennl et al. [2012].

The proposed partially reconfigurable areas are perfectly suited for streaming-based operations with one input port and one output port, like restrictions and aggregations. However, operations like joins need at least two input ports (the two tables), and many SQL operations have to store intermediate results in an external memory, like sorting or the usage of large hash tables. In order to also support such kind of operations, an additional partially reconfigurable area with two input ports, one output port, and access to an external DDR memory is proposed. This partially reconfigurable area supports only the configuration of a single module per time. In Section 8, the architecture of different prototyping systems using a mixture of streaming-based partially reconfigurable areas and such a multiple-port partially reconfigurable area is described.

4. DATABASE MANAGEMENT SYSTEM INTEGRATION

For practical use, it is of the utmost importance to integrate the SQL accelerator into an existing DBMS. The user inputs a query to the *database management system* (DBMS) and the *optimizer* decides which parts of the query are to be executed on the host and which are to be offloaded to the FPGA (see Figure 4). The optimizer has statistics on the requested data and tables and has the knowledge about the feasibility and limitations to execute certain operators on the FPGA. If hardware acceleration of a

query or subquery is chosen, the *module assembly and reconfiguration manager* takes care of that portion of the query execution plan and selects the appropriate partially reconfigurable modules along with their correct initialization parameters. The locations in the partially reconfigurable areas are determined and the FPGA will be reconfigured over the so-called *internal configuration access port* (ICAP). The location inside a partial reconfigurable area depends on the needed slot type (only CLBs, CLB + BRAMs, or CLB + DSP slices) and the operator location on the query plan. After the module reconfiguration, the module parameters are sent from the host to the corresponding modules. The accelerator is now ready to process the data. The incoming tables are streamed through the partially reconfigurable area and the results are transferred back to the host. In some cases, iterations over the accelerator board are necessary with intermediate module reconfiguration. As soon as the entire query has been processed, the result is presented to the user.

A possible system architecture is described in the following: multiple hard drives storing the database are attached to the host PC via a SATA interface. Data transfers between host and FPGA are done using DMA support on the host's main memory. The database tables stored on the disks are transferred to the main memory and are streamed from there to the FPGA over the communication interface. After the hardware query processing, the FPGA stores the (partial) results in main memory for further processing.

The limiting factor of this architecture is the communication bottleneck between the disks, main memory, and FPGA. Therefore, to increase the overall throughput, an improved architecture would attach the disks directly to the FPGA, as in Woods et al. [2014], to avoid or allay this bottleneck in future systems. In this case, the data is streamed from the disks immediately into the FPGA. However, a SATA controller must then be implemented on the accelerator board.

In general, this approach is independent of the type of data and control communication interface. Over the control interface, the partial bit files are loaded and configured. The table data to the FPGA and the results from the FPGA are transferred over the data interface. In Dennl et al. [2013], a *PCI-Express* (PCIe) connection to the host is used to implement both interfaces. In Becher et al. [2014], an energy-aware processing platform is presented that utilizes *AXI interfaces* for the communication with two ARM cores (see also Section 8).

The way that data are organized and stored on disk or in the memory is very important for the data processing and has a high impact on the overall performance of the DBMS. Traditionally, databases store the tables row by row. An alternative way of storing the data is column-wise. This latter approach has been investigated in recent years [Stonebraker et al. 2005] and has shown a significant increase in performance for query execution on some workloads. Therefore, our proposed accelerating system supports both types of data processing (see Section 7).

5. THE MODULE LIBRARY

In the following, we present the operator library that has been designed for our hardware accelerator. The highly optimized modules form a very flexible library of reconfigurable hardware modules. Currently, the library consists of modules for processing *restrictions*, including arithmetics (+, -, *), comparators (<, >, =, ≠), and bitwise functions (AND, OR, NOT, XOR, NAND, NOR); *aggregations* (SUM, MIN, MAX, COUNT); *merge-join*; *hash-join*; *sort*; and a module that reorders, removes, or adds attributes within a tuple. All modules have a uniform interface. The restriction and aggregation modules are described in detail in Dennl et al. [2012] and Dennl et al. [2013], the merge-join, sort, and reorder module in Becher et al. [2014]. Most modules can be used likewise for row- and column-based processing. Only a few modules are optimized for processing data column-wise by using a late materialization strategy (see Section 7.3).

5.1. Restriction

Restriction denotes a group of modules that can handle most of the operations specified in the WHERE clause of an SQL query. This includes arithmetic, comparison, and logical operations, which are implemented in several modules, one for each operation. The positions of the operands and of the result are configurable. Most modules also support a configurable constant as one of the operands. We designed modules for comparison, bitwise operations, and arithmetic operations, as well as a module for string comparison. For more information, see Dennl et al. [2012].

The restriction modules only mark the tuples, but do not remove them from the stream. This is left to a module called *Reorder*, which will be introduced and described in detail in Section 5.3.

5.2. Aggregation

Aggregations like SUM() or MAX() are operations to compute statistical results for multiple tuples of a table. In this work, we consider only the following aggregation functions, because they are very common in databases: SUM(), MIN(), and MAX(). Optionally, aggregations can be combined with a GROUP BY clause to compute aggregated values for groups of tuples, which are defined by the values of one or more attributes specified in the clause. The difference between *single-column* and *multicolumn grouping* is in fact a minor one, because technically multicolumn grouping only results in a higher number of groups, and the grouping attribute is just a compound key. For aggregations without grouping, only the position of the affected attribute inside a tuple and chunk is needed. As soon as the entire table has been processed, the aggregated value has also been computed and can be appended directly to the end of the stream.

If the aggregation is combined with grouping, more than one value must be computed. We need one for each group. Two different approaches can be used to support grouping, each having different advantages and drawbacks.

The first possibility to compute grouped aggregations is the use of hash functions to compute memory addresses for the values of each group. For module implementation, the only memory type that is currently available in partially reconfigurable modules is Block-RAM. Partial reconfiguration allows to use different memory configurations, for example, smaller address ranges but wider data types (important for SUM()), or larger address ranges but smaller data types (sufficient for, e.g., MAX()). With the help of partial reconfigurability, it is also possible to provide a whole bunch of different hash functions for different situations. A big advantage of the hash-based approach, however, is that no tuple sorting is needed; that is, the tuple stream can be processed in the given order.

The second approach is a sorting-based approach, where the tuples of the input table are sorted by their respective group keys. They either are presorted in the table or can be sorted on the FPGA by using the modules *SortLine* and *SortTree* (see Section 5.6). After sorting, one group arrives after the other. Thus, as soon as one group has been streamed through, the aggregated value is completed and is sent on along with the group. This way, only one value must be stored at a time, which completely annihilates the necessity of Block-RAM inside partially reconfigurable modules. Furthermore, the limitation of group counts does not exist anymore. However, the biggest drawback of this approach is sorting. Sorting large amounts of data inside FPGAs is a very resource-intensive task. For more details about the sorting-based approach, see Dennl et al. [2013].

5.3. Reorder

In Section 3, it was shown that tuples that do not fit on the width of the partially reconfigurable area must be cut into chunks. Attributes may be a part of a chunk,

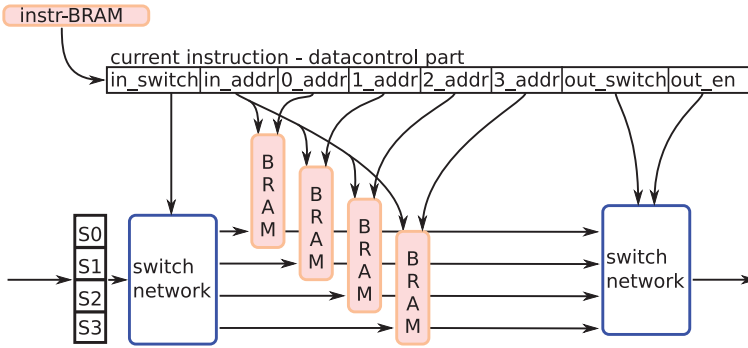


Fig. 5. Concept and structure of a *Reorder* module with a 128-bit interface—data-path control part. Chunks are cut into the 32-bit-wide slices S0, S1, S2, and S3, respectively. After passing through a switching network that reorders these slices according to an input signal provided by the current instruction, each slice is stored in a local 32-bit-wide Block-RAM. The current instruction holds four individual addresses to read data back from the Block-RAMs, and another switching network allows one to change the slice order once again. An out_en signal indicates whether the data produced by the current instructions is valid or not.

but may also use more than one chunk, as is often the case with string attributes. A reordering of the attributes inside of tuples is needed to move attributes to the positions where subsequent modules can process them. Modules also need spare chunks to insert intermediate results or even attribute duplication, for example, to complete a join operation with tuple size greater than one chunk (see Section 5.4). For these tuple manipulations, we introduce a new module that reorders the attributes of tuples. In contrast to prior work such as the stream permutation work of Milder et al. [2009], we use an instruction-based approach and two switch networks instead of one. This is simple and saving Block-RAMs as these would increase the size of this module and therefore decrease flexibility within partial reconfiguration. Furthermore, due to the instruction-based approach, which also allows for branches and conditional branches based on input data, it not only removes unneeded attributes to save bandwidth for following modules, but also gives the opportunity for data-dependent tuple removal.

To accomplish this, each chunk is cut into equal-width slices, taking a practical size of 32 bits, as most data are aligned to that anyway. Dual-ported Block-RAMs are used to store the slices of all chunks of an incoming tuple, each at an individual address (see Figure 5). A switch network can change the order of the slices before they are written to the Block-RAMs. On the output part, the module can read from four independent addresses and reorder the slices again to build a new valid chunk. This allows one to reorder, duplicate, and remove one or more attributes from the stream as long as they are aligned with the chosen slice width. The addresses and switch networks are controlled by *Very Large Instruction Words* (VLIWs) stored in the instruction Block-RAM. Figure 5 shows how the data is controlled by the data-path control part of the current instruction. These VLIWs allow us not only to process each chunk in just one clock cycle, but also to add control-flow support. This means that each incoming chunk of a tuple has at least one individual instruction. More information about the *Reorder* module can be found in Becher et al. [2014].

5.4. MergeJoin

The join operator is highly important in modern relational databases. We have implemented a fast way of joining two tables using our stream-based architecture to further increase the number of operations supported by our acceleration framework. To avoid high resource consumption, a *merge-join* algorithm is used. It is restricted to

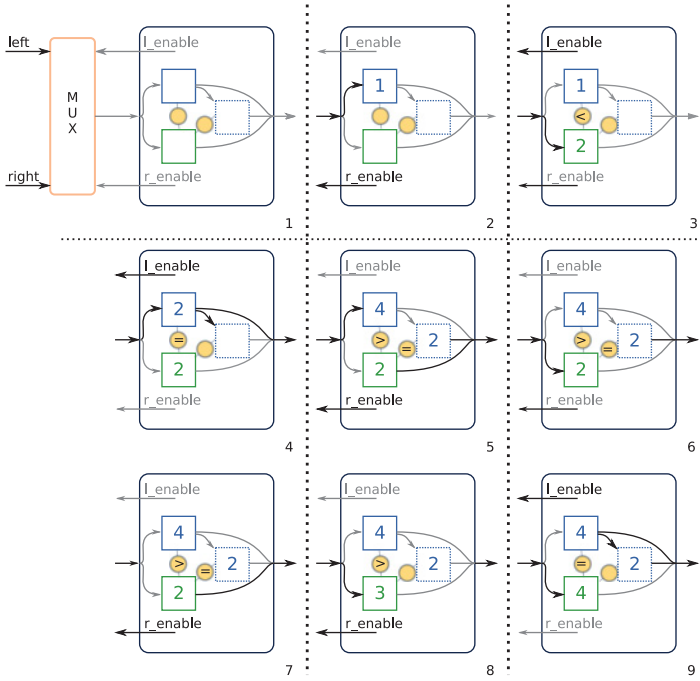


Fig. 6. Example to show how our implementation of a join works. Dark arrows mark the active data path, whereas light ones are switched off. For easier understanding, only the key values of each chunk are shown. After loading the first chunk of each table into built-in registers (Subfigures 1 and 2), they are compared with each other. Since the key value of the left (upper) chunk is smaller, l_enable is set to load the next chunk of the left table. In Subfigure 4, the two key values are now equal, which results in the output of the left chunk and a request for a new one. Also, the key value of the left table is stored in an internal spare register for later use. After the left table chunk has been sent to the output, the right table follows, and the next chunk is requested here as well (Subfigure 5). Subfigure 5 also shows the case of equality with the saved key value, resulting in the output of the saved chunk of the left table. After the chunk from the right table has been forwarded to the output, the inverse behavior can be observed in Subfigures 3 to 8.

one-to-many joins at the moment, but this is known to be the most important case. Figure 6 shows how the join merges two sorted input streams into an output stream by controlling a multiplexer (called left and right in the figure).

With the use of a spare register (dotted squares in Figure 6), our design supports one-to-many joins, as long as one tuple does not consist of more than one chunk. A *Reorder* module as introduced in Section 5.3 that is used afterwards, for example, to strip off no longer needed attributes, can also reconstruct missing chunks from 1:n joins by copying them. This eliminates the one-chunk-only limitation.

For this stream-based merge-join approach to work, the tables must be sorted by the join keys. Today, most of the large tables are sorted anyway by the attribute mostly used in joins, or they are indexed accordingly with the option to read the index in sort-order (e.g., B+-tree). Restrictions may also filter incoming data before they enter the join module. For the resulting smaller amounts of data, we propose a fast *SortLine* module, which will be presented in Section 5.6. Another prerequisite of our approach is that the key attribute must be in the first chunk of a tuple to limit the amount of data stored inside the join module. Once again, a reorder unit can resolve that limitation.

A left outer join outputs the complete left table and adds a tuple from the right table, if a common key value is found. This behavior can be evoked by treating a smaller ($<$)

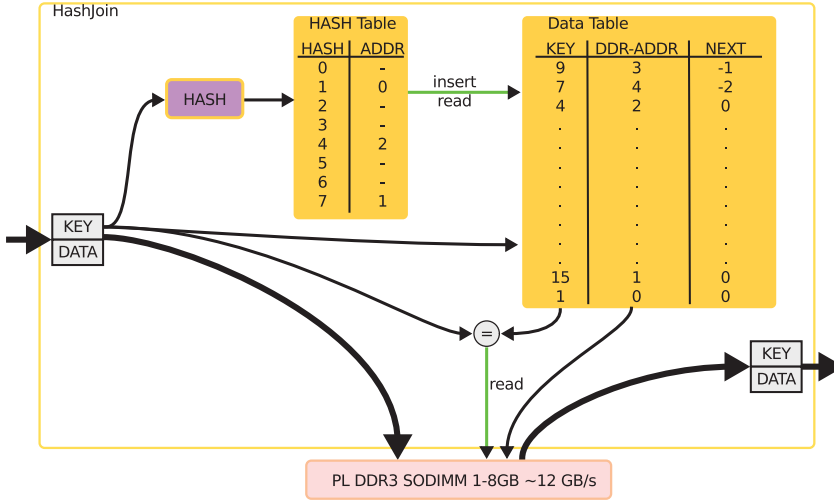


Fig. 7. The *HASHJoin* Module with the *HASH* table and the *Data* table. The keys {1, 15, 4, 9, 7} were inserted in this order.

in the first comparison of Figure 6 also as an equal ($=$), but creating empty chunks instead of forwarding the right table.

5.5. HashJoin

For increasing the flexibility of our database accelerator system, a module for *hash-based joins* was implemented and will be described in this section. Databases using the *Star Schema* on which a single large table is joined with many small tables benefit from the so called *hash-join*. In contrast to the previously introduced *merge-join*, this implementation does not require any correctly sorted tables as inputs. It utilizes *hash tables* for fast lookups of matching join attributes. This might lead to a much faster join operation. One important goal for implementing a hash-join is finding a good *hash function* that minimizes the likelihood of collisions. This can be achieved by selecting a function that will result in evenly distributed hash values over the whole set of resulting hash values. As the most common join attribute is some sort of numeric identifier (ID), a simple *modulo* function is suitable as the hash function and will be used here.

The actual implementation of the *HashJoin* module consists of two entities, namely, *HASH table* and *Data table*. The *HASH table* is a real hash table that holds a bit to indicate if an entry is valid (*hit bit*) and a corresponding address for the *Data table*. The *Data table* stores, depending on its configuration, one whole chunk of data or an address for the *DDR* interface. Furthermore, to allow chaining inside the table, a *NEXT* address that points to the following entry is stored in each entry of the *Data table*. This way, the collisions are resolved as equal hash values lead to multiple, individual entries in the *Data table*. If this *NEXT* address is 0, it is considered the last element of this list. A negative value means that the address counts from the last memory location down to the first one. The value -1 , for example, would be the last entry in the *Data table*. With this addressing scheme, it is possible to have all first entries of these lists at the beginning of the *Data table*. Probing just these values would result in a *GROUP-BY* function if collisions can be disregarded.

Figure 7 shows the block diagram of the implemented *HashJoin* with its connections to the *DDR3* interface and the control signals *read*, *store*, and *get*. The *store* signal

triggers the *Data table* to insert the current *KEY*—which is the *join* attribute—into the list. If an entry is already present at the given address, it is moved to the next free address of the table counted backward (negative address) and the new *KEY* with its corresponding DDR address (*DDR-ADDR*) is inserted. Once the first table is completely streamed, the *get* signal comes into play. It is active if a hashed key from the second table finds a match inside the *HASH table*. The *Data table* will then output the whole list starting at the address stored in the *HASH table* in sequential order after each clock cycle. The equal symbol compares the key from table two with the one stored in the *Data table*. If it matches, the join is successful and the *read* signal is sent to the *DDR* interface to read the data from the address (*DDR-ADDR*) stored inside the *Data table*.

5.6. Sort

Sorting is one of the most processing-intensive parts of a query; thus, it is important to have a fast hardware acceleration. Our dynamic accelerator system is based on a sorter concept described in Koch and Torresen [2011]. It consists of the following two modules:

The *SortLine* is the first sorting module that is responsible for presorting blocks with the size of a few kilobytes. It needs no access to an external memory and fully relies on on-chip distributed RAM and Block-RAM. It cannot sort large datasets, only fixed-sized blocks. The big advantage of the *SortLine* is that it is fully pipelined, so it can process one chunk each cycle. This module consists of multiple stages. Stage i requires memory to store 2^i chunks. A *SortLine* of k stages can sort blocks of 2^k chunks.

The sorted blocks can then be processed by the second sorting module, the *SortTree*. The *SortTree* merges the presorted blocks to a fully sorted table by using external memory. The external memory is necessary to be able to sort large blocks of data. The *SortTree* module is not fully pipelined. It may need multiple iterations i to sort the entire data. It consists of an n -depth tree that sorts 2^n blocks of any size. For example, a three-deep *SortTree* feed with 64kB sorted blocks from the *SortLine* produces 512kB blocks in a single iteration. In the second iteration, the result would be sorted in blocks of 4MB.

5.7. BitVector

To process data using late materialization (see Section 7.3 for details), a list of required rows must be returned from the accelerator to the CPU. This could be a list of primary keys, but that would consume too much bandwidth—not only on the return path, because the primary key must have been sent to the accelerator in the first place. As all columns are stored in the same order, one bit per row is enough to provide the information.

The implemented module looks for the result of the restriction in each input tuple and saves it as 1 bit of a vector. Once 128 values are stored, an output chunk is generated. When an end-of-file signal is received, the remaining bits of the current vector are filled with zeroes and the vector is written to the output. The input tuple itself is discarded by the module.

5.8. ColumnJoin

The *ColumnJoin* module is used for column-based query processing only. As columns are stored separately from each other, there must be a way to rejoin them again, for cases where multiple columns are required to execute an operation, that is, for materialization of (parts of) tuples.

The *ColumnJoin* module does an implicit join of multiple columns without needing a key. The columns are joined $1 : 1 : 1 : \dots$ based on the positions. That is, the n th

value of the first column is joined with the n th value of the second column. It is not allowed that columns are sorted or filtered before joining; otherwise, the orderings would be inconsistent. The module starts to emit data to the remainder of the partially reconfigurable area after one block of each column has arrived. The module is placed at the start of the partially reconfigurable area and needs memory to store two blocks of each column. This allows to produce output while at the same time receiving input.

6. A CALCULUS FOR PERFORMANCE ASSESSMENT

In this section, a calculus for the performance assessment of query processing in our system is introduced. The goal and purpose of this calculus is to be able to evaluate which parameters influence the achievable throughput and latency of query processing in which way. These parameters could be clock frequency, number of modules, bit-widths, and other hardware parameters.

In order to analyze the throughput of the acceleration system, the processing delay d of a chunk (measured in nanoseconds) and the throughput D of each module as well as that of the input and output interfaces are determined. Due to our pipelined architecture, most of the modules have a constant throughput of one chunk per clock cycle. By taking the bus width of a partially reconfigurable area $width_{pa}$ and the clock frequency f_{clock} , a throughput D_{pa} of $width_{pa} \cdot f_{clock}$ can be achieved.

In addition to the delay of chunks and throughput of each module, the total execution time t_{exec} also depends on the number of tuples T to process and on the size C of a tuple (measured in chunks). These two values are not constant and might be different from module to module. Thus, we use a 4-tuple (d, D, T, C) to describe the behavior each module. It depends on the intrinsics of the module as well as the previous component in the pipeline, or in case of the join, on the two inputs. The first component is always the input interface from the host. The total execution time can be therefore calculated from the 4-tuple of the last module in the chain:

$$t_{exec} = d_{last} + \frac{C_{last} \cdot T_{last} \cdot width_{pa}}{D_{last}}. \quad (1)$$

The path from the main memory of the host to the FPGA and back cannot be exactly captured by formulas. The driver, cache hierarchies, interrupts, multiple CPU cores, and components sharing a bus all together make the system too complex and indeterministic. For this reason, a maximum throughput is measured on the system using a single empty partially reconfigurable area. It must be taken into account that the input and output streams may share one medium.

6.1. Performance Assessment of Library Modules

For each module type, the formulas to calculate the 4-tuple of the output $(d_{out}, D_{out}, T_{out}, C_{out})$ are presented in Table I. As input vector $(d_{in}, D_{in}, T_{in}, C_{in})$, the output vector of the previous module in the chain is used. The join modules use two input vectors $(in1, in2)$, one for each input stream.

For our prototypical implementation (see Section 8), the latency l of the corresponding module in clock cycles is set for restrictions and aggregations to $l_{Restr} = l_{Aggr} = 3$, for the *Reorder* module to $l_{Reorder} = 6$, for the *MergeJoin* module to $l_{MergeJoin} = 6$, for the *ColumnJoin* module to $l_{ColumnJoin} = 5$, and for the *BitVector* module to $l_{BitVector} = 4$. The latency of the k_{line} -stage *SortLine* module can be calculated as follows: $l_{SortLine} = 2^{k_{line}+1} + 3 \cdot k_{line} - 2$. The *Reorder* module filters tuples that are marked for removal from previous restrictions with a selectivity $0 \geq s \geq 1$ and truncates unneeded attributes. The latter has a high impact on the number of output chunks C_{new} .

Table I. The Formulas for Delay d , Throughput D , Tuple Count T , and Size C for a Selection of Modules

	Restriction	Aggregation	SortLine
d_{out}	$d_{in} + \frac{l_{Restr}}{f_{clock}}$	$d_{in} + \frac{l_{Aggr}}{f_{clock}}$	$d_{in} + \frac{l_{SortLine}}{f_{clock}}$
D_{out}	$\min(D_{pa}, D_{in})$	$\min(D_{pa}, D_{in})$	$\min(D_{pa}, D_{in})$
T_{out}	T_{in}	T_{in}	T_{in}
C_{out}	C_{in}	C_{in}	C_{in}
	Reorder	HashJoin	SortTree
d_{out}	$d_{in} + \frac{l_{Reorder} + C_{in}}{f_{clock}}$	$d_{in1} + d_{in2} + \frac{l_{HashJoin}}{f_{clock}}$	$d_{in} + \frac{i \cdot l_{SortTree}}{f_{clock}} + \frac{S}{\min(D_{mem}, D_{in})} + \frac{S \cdot (i-1)}{\min(D_{io}, D_{mem})}$
D_{out}	$\min(s \cdot \frac{C_{new}}{C_{in}} \cdot D_{in}, D_{pa})$	$\min(D_{HashJoin}, D_{pa})$	$\min(D_{pa}, D_{mem})$
T_{out}	$s \cdot T_{in}$	$m_j \cdot T_{in1} \cdot T_{in2}$	T_{in}
C_{out}	C_{new}	$C_{in1} + C_{in2}$	C_{in}
	MergeJoin	ColumnJoin	BitVector
d_{out}	$\max(d_{in1}, d_{in2}) + \frac{l_{MergeJoin}}{f_{clock}}$	$d_{in} + \frac{l_{ColumnJoin} + \frac{col \cdot size_{block}}{width_{pa}}}{f_{clock}}$	$d_{in} + \frac{l_{BitVector} + width_{pa} \cdot C_{in}}{f_{clock}}$
D_{out}	$\min(D_{part1}, D_{part2}, D_{pa})$	$\min(\frac{width_{pa}}{col \cdot width_{attr}} \cdot D_{in}, D_{pa})$	$D_{in} \cdot cr$
T_{out}	$m_j \cdot T_{in1} \cdot T_{in2}$	T_{in}	T_{in}
C_{out}	$C_{in1} + C_{in2}$	1	$\frac{1}{width_{pa}}$

The hash-join performance assessment is simplified. The hash-join latency $l_{HashJoin}$ and throughput $D_{HashJoin}$ depend on the collision rate, the latency for resolving collisions, and the latency for reading values from the external DDR memory. The selectivity m_j of the join can be calculated with $m_j = T_{out} / (T_{in1} \cdot T_{in2})$.

The *SortTree* module processes the data with i iterations and the latency for one iteration $l_{SortTree}$ depends on the implementation of the module itself and the controller of the external memory. D_{mem} denotes the available data throughput to the external memory. The data rate D_{part1} for calculating the overall throughput of the *MergeJoin* can be calculated as follows:

$$D_{part1} = m_j \cdot T_{in2} \cdot D_{in1} \cdot \frac{C_{in1} + C_{in2}}{C_{in1}}. \quad (2)$$

The data rate D_{part2} can be calculated in a similar way.

The *ColumnJoin* module joins col columns and $size_{block}$ is the size of a single block sent from the CPU. The width of the join attribute is $width_{attr}$. The compression rate cr of the *BitVector* module can be calculated with $cr = 1 / (width_{pa} \cdot C_{in})$.

6.2. Performance Assessment of an Example Query

To calculate the execution time of a full query, the formulas for the individual modules must be combined. This is done step by step starting from the input. This output of one module is the input for the next one. The latencies are simply summed up and give the time from the first tuple moving to the accelerator to the first tuple arriving at the output. Modules in the partially reconfigurable area not only add latency but also may change the amount of data and the rate of data output. A simple example query is introduced in Listing 2 to show how the execution time calculation works.

The table has 365 entries (T_{days}) and each row has a size of 26 bytes, requiring two chunks (C_{days}) for a 128-bit partially reconfigurable area architecture. The query can be processed by a *Restriction* module followed by a *Reorder* module. The steps to calculate the 4-tuple for each module are shown in Table II. The last column can then be used to

Table II. The Steps for Composing Delay d , Throughput D , Tuple Count T , and Size C for the Example Query

	After Input IF (0)	After Restriction (1)	After Reorder (2)	After Output IF (3)
d	d_{if}	$d_{if} + \frac{l_{Restr}}{f_{clock}}$	$d_{if} + \frac{l_{Restr} + l_{Reorder} + C_{days}}{f_{clock}}$	$2 \cdot d_{if} + \frac{l_{Restr} + l_{Reorder} + C_{days}}{f_{clock}}$
D	D_{if}	$\min(D_{pa}, D_{if})$	$\min(s \cdot \frac{C_{new}}{C_{days}}, \min(D_{pa}, D_{if}), D_{pa})$	$\min(D_{if}, \min(s \cdot \frac{C_{new}}{C_{days}}, \min(D_{pa}, D_{if}), D_{pa}))$
T	T_{days}	T_{days}	$s \cdot T_{days}$	$s \cdot T_{days}$
C	C_{days}	$C_{days} = 2$	C_{new}	C_{new}

```

SELECT day.in_month, month, year
FROM days
WHERE day.in_week = 1;

```

Listing 2. Example query requesting all Mondays in a date table.

calculate the execution time:

$$t_{exec} = d_3 + \frac{C_3 \cdot T_3 \cdot width_{pa}}{D_3} \quad (3)$$

$$= 2 \cdot d_{IF} + \frac{l_{Restr} + l_{Reorder} + C_{days}}{f_{clock}} + \frac{C_{new} \cdot s \cdot T_{days} \cdot width_{pa}}{\min(D_{if}, \min(s \cdot \frac{C_{new}}{C_{days}}, \min(D_{pa}, D_{if}), D_{io}))}. \quad (4)$$

With a frequency f_{clock} of 125MHz, $width_{pa}$ of 128 bits, interface throughput of 0.1GB/s, and interface delay of $5\mu s$, this results in a total execution time t_{exec} of $127\mu s$.

7. ROW-BASED VERSUS COLUMN-BASED PROCESSING

Traditionally, databases store the tables row by row. However, for many queries, a significant increase in performance is achieved if the data is processed column-wise (see, for instance, Stonebraker et al. [2005]). In the following, different strategies are proposed to execute an example query based on the TPC-DS benchmark [Council 2012] (see Listing 3). This query is used as a representative example, which will be used for the remainder of this article.

```

SELECT dt.d_year, dt.d_moy,
  sum(store_sales.ss_ext_sales_price)
FROM date_dim dt, store_sales
WHERE dt.d_date_sk = store_sales.ss_sold_date_sk
AND dt.d_moy = 12 AND dt.d_year = 2000;

```

Listing 3. An SQL query based on the TPC-DS benchmark is used for evaluation.

7.1. Row-Based Query Processing

The strategy for processing the query with traditional row-based storage is discussed first. In this setup, the input to the first partially reconfigurable area is always a full table (or the intermediate result of a prior run). This may be a huge amount of data; thus, the idea is to first remove as much data as possible. This is done by applying all restrictions that affect only the current input table and then using a *Reorder* module (introduced in Section 5) to apply the filter and to remove all columns not needed for the remainder of the query. This can easily result in a reduction of the input data by a factor of 100 or more. The following join has only to process the remaining data. The query plan for row-based processing is given in Figure 8.

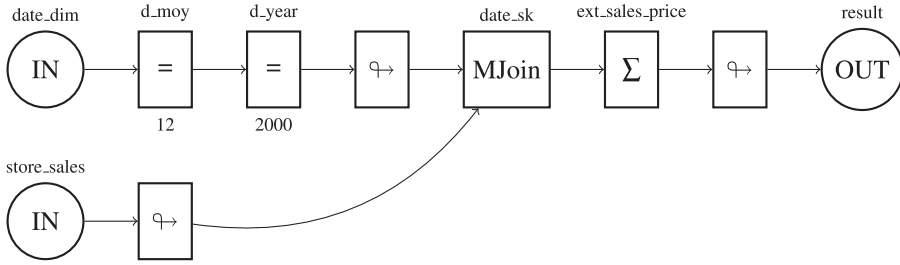


Fig. 8. Plan for row-based execution of the query depicted in Listing 3. Circles represent data sources or sinks. The modules are displayed using the symbols. The symbol $\lceil \leftrightarrow \rceil$ denotes the *Reorder* module. Arrows represent data flow. Involved tables/attributes/constants are shown above or below the nodes.

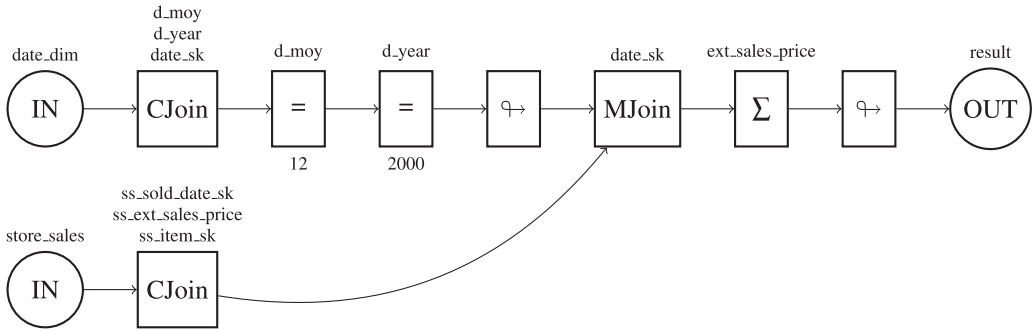


Fig. 9. Plan for column-based execution of the query depicted in Listing 3 using early materialization. Circles represent data sources or sinks. The modules are displayed using the symbols. The symbol $\lceil \leftrightarrow \rceil$ denotes the *Reorder* module. Arrows represent data flow. Involved tables/attributes/constants are shown above or below the nodes.

7.2. Column-Based Query Processing: Early Materialization

The strategy for column-based storage with early materialization is similar to the row-based approach. The idea of early materialization is to join the columns at the beginning of the processing to reconstruct the (parts of the) tuples that are required for the execution of a particular query [Abadi et al. 2008]. After that, all modules developed for row-based storage can be used.

Instead of loading the full table and then filtering with the *Reorder* module, we only load the required columns using the *ColumnJoin* module introduced in Section 5. One *Reorder* module can be skipped, as its only job was removing columns. The resulting plan is shown in Figure 9.

7.3. Column-Based Query Processing: Late Materialization

Late materialization [Abadi et al. 2008] is an approach that tries to execute the query with the columns retrieved from storage as far as possible and only materialize tuples when required for output to the user. Thus, data (in the form of columns) is loaded as late as possible, which minimizes the transfer data to the board. Using the current architecture, this results in many inexpensive subexpressions executed one after another by the accelerator.

Late materialization also requires random access to the data, because other attributes of the tuple at hand can be stored at different positions in their column structure. That cannot be achieved with a streaming accelerator. The accelerator, therefore, generates bit-vectors that contain a list of the required rows. The CPU gathers these

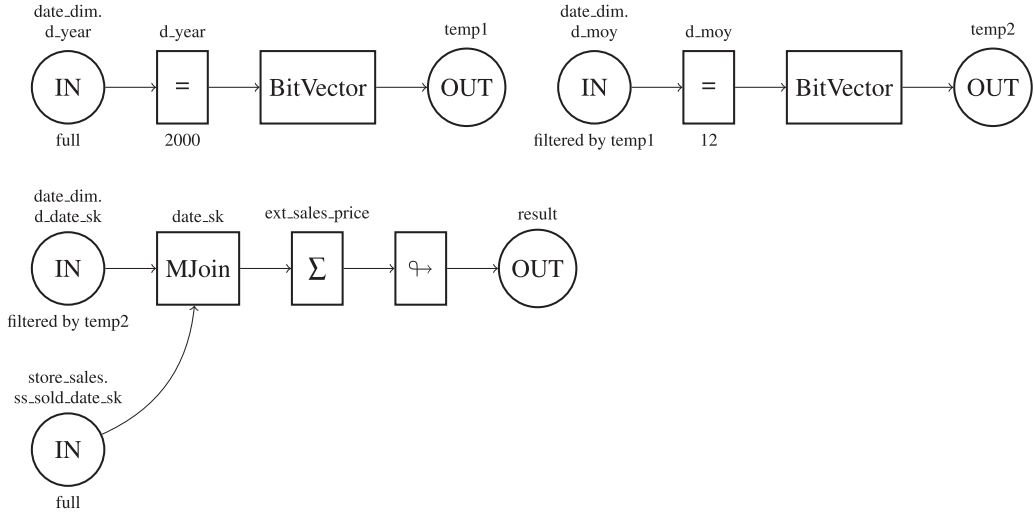


Fig. 10. Plan for column-based execution of the query depicted in Listing 3 using late materialization. Circles represent data sources or sinks. The modules are displayed using the symbols. The symbol $\leftrightarrow$ denotes the *Reorder* module. Arrows represent data flow. Involved tables/attributes/constants are shown above or below the nodes. The figure only shows the parts executed on the accelerator.

values and streams them back to the accelerator. This happens in parallel to the execution on the accelerator, so the overhead is acceptable.

Figure 10 shows a possible execution plan for the example query using late materialization. There are many different ways to execute it. Which to prefer highly depends on the data and the available resources. One scenario is to load the three different independent query plan parts depicted in Figure 10 sequentially on the query processing system by using partial reconfiguration and storing the intermediate results in the main memory. A mixture of early and late materialization may be appropriate to balance storage I/O and the number of executed subqueries.

8. PROTOTYPES

We have prototyped our partially reconfigurable query accelerator engine on a Xilinx ML605 board, which includes a Virtex-6 XC6VLX240T FPGA and two boards based on the Xilinx Zynq SoC (Zedboard and Xilinx ZC706 board). For reasons of flexibility, we have decided to segment the reconfigurable fabric into a static system consisting of four partially reconfigurable areas, a PCIe controller for the ML605 board and AXI controllers for the Zynq boards, and a hardware part of the reconfiguration manager to configure the system (see Figure 11). The latter is responsible for loading the respective partially reconfigurable modules over ICAP, for configuring the module parameters, and for configuring the interface between the communication interface (PCIe or AXI) and the partially reconfigurable areas. Four partially reconfigurable areas have been chosen to be able to process four queries at the same time. Alternatively, the partially reconfigurable areas can be concatenated to support queries with higher processing depth.

Finally, each partially reconfigurable area is divided into 16 *slots* with each slot having a height of 40 CLBs and a width of 2 CLBs, or a 1-CLB and 1-DSP/BRAM unit (see Figure 12). Thus, one slot contains 640 or 1,280 flip-flops and 320 or 640 LUTs. Table III shows the resource usage of the different partially reconfigurable modules on Virtex-6. Furthermore, each reconfigurable area has a 128-bit data bus.

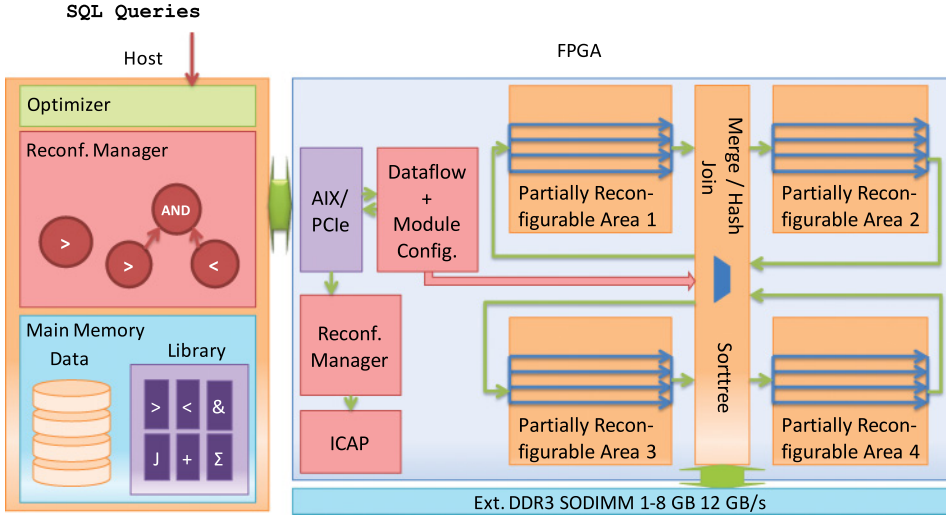


Fig. 11. Architecture of our prototypes. On the left-hand side, the software program's optimizer, the reconfiguration manager, and the host memory containing the database and the operation library are depicted. On the right-hand side, the FPGA-based architecture including the partially reconfigurable areas, the communication interface, and the control modules are shown.

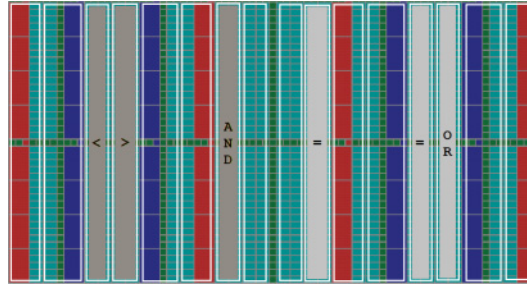


Fig. 12. Slot-based partitioning of a partially reconfigurable area on Virtex-6 with 16 slots (white-border rectangles). In the background, the different FPGA resources are shown (cyan = CLBs, blue = BRAMs, red = DSPs).

Table III. List of Developed Partially Reconfigurable Modules with Resource Use and Throughput (the Modules That Have an N/A Entry in the #Slots Column Are Mapped to the Central Partially Reconfigurable Area with Memory Access)

Module	Operator Coverage	#Slots	Max. Throughput		
			ML605	Zedboard	ZC706
Restriction	+, -, ×, <, >, =, ≠, AND, OR, NOT, XOR, ...	2	2GB/s	2.4GB/s	2.8GB/s
Aggregation	SUM(), MIN(), MAX(), COUNT()	2	2GB/s	2.4GB/s	2.8GB/s
Reorder	Reorder attributes of a tuple	4	2GB/s	2.4GB/s	2.8GB/s
Join	Merge join, Hash join	n/a	2GB/s	2.4GB/s	2.8GB/s
Sort	Sortline for 2kB data	16	2GB/s	2.4GB/s	2.8GB/s
Sort	SortTree	n/a	2GB/s	2.4GB/s	2.8GB/s
Column-based	BitVector, CoulmnJoin	2	2GB/s	2.4GB/s	2.8GB/s

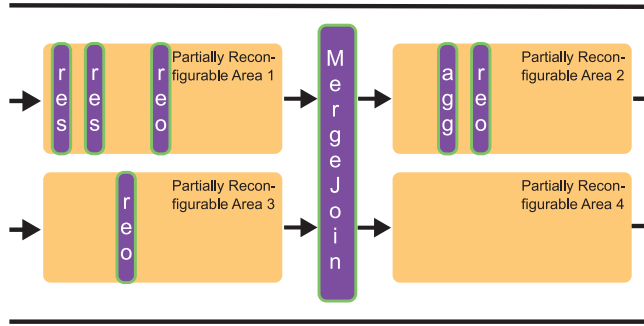


Fig. 13. The query from Listing 3 requires three *Reorders* (reo), two *Restrictions* (res), one *MergeJoin*, and one *Aggregation* (aggr) module. In this figure, we see how the modules are placed. The table *date_dim* is streamed to reconfigurable area 1, where the first and second restriction filter for $d_moy = 11$ AND $d_year = 2000$ reside. Before the two data streams enter the *MergeJoin*, one *Reorder* module before each join input strips off not needed attributes. The joined table enters the aggregation to sum *ext_sales_price* and another *reorder unit* moves the attributes so the output matches the selected format of the query.

The system operates at 125MHz on the Virtex-6, 150MHz on the Zedboard, and 175MHz on the ZC706, which delivers a total throughput of 2GB/s, 2.4GB/s, and 2.8GB/s per reconfigurable area, respectively. Therefore, loading all four reconfigurable areas with independent queries leads to a theoretical maximum throughput of 8GB/s, 9.6GB/s, or 11.2GB/s, respectively. The PCIe Gen2 bus with four lanes can only provide a net transfer rate of 1.7GB/s for each direction; that is, the bus is clearly the bottleneck in this setup. The SoCs consist of a dual-core ARM Cortex A9 clocked at 666MHz and an FPGA fabric. The partially reconfigurable areas are connected to the ARM and the DDR memory via AXI-DMA cores with 64-bit data width. The two high-performance ports run at a clock frequency of 175MHz. One general-purpose AXI-lite bus is used to control the DMA cores. On the software side, we use a *Linaro-Ubuntu 14.02* distribution running a *Linux 3.12.0-xilinx* kernel.

9. EXPERIMENTAL RESULTS

In the following, we compare our prototypes with software implementations. Furthermore, we show that our performance assessment calculus presented in Section 6 is quite accurate. The overhead introduced by partial reconfiguration is shown, and with the help of the performance assessment, the processing time of different processing strategies (row-wise or column-wise) is shown for different table sizes.

As a representative example, an accelerator for the query depicted in Listing 3 in Section 7 is used. In order to map the whole query to our accelerator system, we need to instantiate two *Restriction* modules, one *Aggregation* module, three *Reorder* modules, and a *MergeJoin* module. The corresponding placement of the modules in the partially reconfigurable areas of our platform is shown in Figure 13.

9.1. Comparison to Software-Only Approaches

We compare our proposed prototypes with software-only solutions running on an x86-based server and on the ARM cores of the ZedBoard. Furthermore, a comparison of the different prototypes is presented. For the ARM and x86-based software-only configuration, we use a MySQL installation each with INMEMORY tables to neglect the I/O traffic from the hard disc or SD card, respectively. Our x86-based server system is running *Arch Linux* with a 3.7.9 kernel on an Intel Core i7-3770 CPU clocked at 3.4GHz and 16GB DDR3 memory. The ML605-based system is connected to our x86 server over PCIe Gen2 using four lanes. Note that the throughput and latency of all

Table IV. Processing Time t_{exec} and Energy Consumption of Queries in Listing 4 for a Database Size of 1GB Scale on Different Hardware Configurations

Query	Zynq-Based System						x86-Based System		
	Configuration		Softw.-Only	Accelerated			Software-Only	Accel.	
	Board	Join Type	MySQL	Analyzed	Measured		MySQL	Analyzed	
Q1	ZedBoard	Merge Join	6,760ms	345.6ms	347.3ms	0.13J	380ms	4.82J	304.4ms
Q1	ZC706	Hash Join	6,760ms	155.22ms	158.77ms	0.267J	380ms	4.82J	310.1ms
Q2	ZC706	Hash Join	6,770ms	176.49ms	176.67ms	0.297J	380ms	4.82J	341.6ms
Q3	ZC706		80ms	3.79ms	3.86ms	0.005J	10ms	0.13J	6.28ms
Q4	ZC706		80ms	3.79ms	3.86ms	0.005J	10ms	0.13J	6.28ms

streaming-based modules that can be combined to a pipeline and loaded into the streaming-based partial areas are absolutely deterministic. Therefore, we choose only some selected scenarios to back up our performance assessment. In addition to the processing of the query in Listing 3 by using a merge join on the Zedboard, we process the query and some derivations, shown in Listing 4, on the ZC706 board by using the hash-join, which utilizes the external memory.

Table IV shows the processing time t_{exec} of the queries in Listing 4 on our proposed prototypes calculated using the throughput analysis presented in Section 6 and real measured values. Furthermore, the execution time of software-only approaches using MySQL on the x86-based server and the embedded ARM core are given. All these results are obtained for the TPC-DS database with the scale factor of 1GB. For the throughput analysis, we measure an AXI-interface throughput of 1.2GB/s for the Zedboard and 2.77GB/s for the ZC706 board. For the PCIe Gen2 connection with four lanes, we assumed a throughput of 1.7GB/s. To emphasize the benefits of our approach regarding energy consumption, we include measurement in each individual query in Table IV. For determining the energy consumption, the system power on query processing and the idle power are measured. The shown energy values include only the additional energy for query processing. Note that the idle power of the x86 system is 32.7W compared to 12.4W for the ZC706 board and 3.6W for the Zedboard.

Q1: **SELECT** dt.d_year, dt.d_moy, **SUM**(ss.ss_ext.sales.price) **FROM** date_dim dt, store_sales ss **WHERE** dt.d_date_sk = ss.ss_sold_date_sk **AND** dt.d_moy = 12 **AND** dt.d_year = 2000;
Q2: **SELECT** dt.d_year, dt.d_moy, **SUM**(ss.ss_ext.sales.price) **FROM** date_dim dt, store_sales ss **WHERE** dt.d_date_sk = ss.ss_sold_date_sk **AND** dt.d_year = 2000;
Q3: **SELECT SUM**(d.fy_quarter_seq) **FROM** date_dim dt **WHERE** dt.d_moy = 12 **AND** dt.d_year = 2000;
Q4: **SELECT * FROM** date_dim dt **WHERE** dt.d_moy = 12 **AND** dt.d_year = 2000;

Listing 4. SQL queries for evaluation based on the query in Listing 3 (Q1 is the query in Listing 3).

It can be seen that both accelerated embedded Zynq-based systems and the ML605 prototype are faster than the MySQL-based x86-system even if a reconfiguration overhead of 1.88ms (see Section 9.2) is included in order to load the partially reconfigurable modules initially. However, an optimized C++ solution on the x86 platform would outperform our solution due to a much higher memory bandwidth (see Becher et al. [2014] for details). It can be seen that for Zynq targets, our presented analytic formula-based query execution time determination is quite accurate. Finally, the experimental results give evidence that the proposed architecture, especially on the low-power Zedboard, may reduce the amount of consumed energy significantly compared to the energy needed by an in-memory database system running on an x86-based server at equal throughput.

Table V. Configuration Time and Throughput for the PCIe and AXI Interface (the ICAP Interface Is the Limiting Factor for the Partial Reconfiguration)

Modules			PCIe or AXI Interface	
Name	No. Slots	Size	Conf. Time	Throughput
1 × restriction	2	47kB	118 μ s	400MB/s
1 × reorder	4	132kB	330 μ s	400MB/s
8 × restrictions	16	376kB	940 μ s	400MB/s
(full partially reconfigurable area)				
32 × restrictions	64	1,504kB	3.76ms	400MB/s
(4 partially reconfigurable areas)				
Query in Listing 3 (See Figure 8)	18 + MergeJoin	750kB	1.88ms	400MB/s

9.2. Overhead Introduced by Partial Reconfiguration

The partial dynamic reconfiguration process utilizes one of the configuration ports of the FPGA. In our approach, we are using the *internal configuration access port* (ICAP), which makes the configuration memory accessible from the FPGA fabric. The ICAP interface of the Xilinx Virtex-6 and Zynq has a data width of 32 bits and can be clocked up to 100MHz, which results in a maximal configuration throughput of 400MB/s. Furthermore, there exist some approaches to overclock the configuration interface in order to achieve a higher configuration speed [Claus et al. 2010].

The second parameter, which influences the configuration speed, is the interface between the host and the FPGA board. Since the partial bit files are stored on the filesystem of the host, the bit files have to be transferred to the FPGA. The transfer rates of the PCIe interface of the ML605 board and the AXI interfaces of the Zynq boards are higher than the 400MB/s of the configuration interface. Therefore, the bottleneck is the limitation of the ICAP. Table V presents the module loading time of different module sizes.

9.3. Overall Performance Assessment

In order to evaluate the overall performance of our approach, the database scheme and corresponding table data from the TPC-DS benchmark [Council 2012] are used. As a representative query, the query in Listing 3 is taken for this analysis. The performance estimation is done by using our model for performance assessment, described in Section 6, with the performance models of the corresponding modules, described in Section 5.

For this evaluation, we use our prototype, based on the Xilinx Virtex-6 ML605-board (see Section 8). For the interface throughput, we estimate a realistic throughput of 1.7GB/s, whereas the throughput of the on-board DDR3 interface is 12GB/s. The partially reconfigurable areas are 128 bits wide and are clocked with 125MHz, which results in an overall throughput of 2GB/s for each partially reconfigurable area.

The performance of the query in Listing 3 is analyzed by using a row-based (see Section 7.1) and two different column-based processing approaches. The first one is early materialization (see Section 7.2) and the second is late materialization (see Section 7.3). Each of these processing scenarios is further evaluated by using sort-merge-join and hash-join approaches. The sort-merge-join approaches are subdivided into two cases. The first case assumes that the tables are presorted, so only the merge-join is processed on the FPGA. In the second case, the tables must be sorted on the FPGA before the merge-join can be processed. The results are shown in Table VI for different benchmark database scales and depicted in Figure 14.

It can be seen that the processing time rises linearly with the database size. Furthermore, the significant improvement by utilizing column processing can also be seen. The

Table VI. The Results for the Performance Assessment of the Query Shown in Listing 3 and Different TPC-DS Benchmark [Council 2012] Database Scales for Row-Based Processing (RS) and Column-Based Processing by Using Early Materialization (EM) and Late Materialization (LM)

Database Scale	Processing Time in Seconds							
	1GB	100GB	300GB	1TB	3TB	10TB	30TB	100TB
Merge/RS	0.30	29.82	89.46	298.17	894.49	2,981.65	8,944.88	29,816.34
Sort-Merge/RS	0.66	68.86	206.56	688.50	2,134.58	7,115.30	22,036.93	73,456.62
Hash/RS	0.31	30.38	91.13	303.74	911.20	3,037.35	9,111.96	30,373.28
Merge/CS-EM	0.02	2.30	6.91	23.04	69.12	230.40	691.19	2,303.99
Sort-Merge/CS-EM	0.09	13.82	41.47	138.24	483.84	1,612.80	5,529.56	18,431.92
Hash/CS-EM	0.02	2.38	7.15	23.84	71.51	238.35	715.05	2,383.51
Merge/CS-LM	0.02	2.30	6.91	23.04	69.12	230.40	691.20	2,303.99
Sort-Merge/CS-LM	0.10	14.64	43.91	146.37	508.23	1,694.12	5,773.51	19,245.09
Hash/CS-LM	0.02	2.38	7.15	23.84	71.51	238.35	715.05	2,383.51

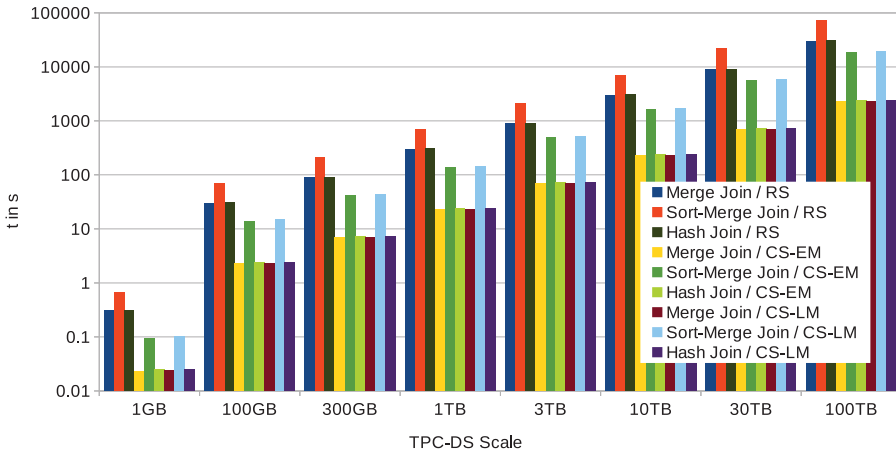


Fig. 14. The results for the performance assessment of the query shown in Listing 3 and different TPC-DS benchmark [Council 2012] database scales for row-based processing (RS) and column-based processing by using early materialization (EM) and late materialization (LM). This figure shows the results from the merge-join with sorting (sort-merge-join) and the processing with presorted tables with the merge-join (merge-join) as well as the processing time by utilizing the hash-join module.

difference between early and late materialization is almost negligible due to the very small dimension table *date_dim*. The TPC-DS benchmark scales only the fact table *store_sales* and not the dimension tables. Furthermore, by applying the restrictions on the dimension table, only 30 entries are left for the join, which is further independent of the database scale. By comparing the different join variants, the hash-join has almost the same processing time as the merge-join without sorting. Sorting introduces a significant latency increase. Furthermore, for sorting large tables, a huge amount of on-board memory is needed. For example, for processing the fact table of the 1TB database, almost 100GB on-board DDR3 memory is needed. This huge memory is not needed by using the hash-join or presorted tables combined with the merge-join. This means that with our current system, we are able to process the query at all database scales, even the 100TB, if the host system has the corresponding storage. The different processing possibilities of the query can be analyzed before the processing, and an optimizer can choose the configuration with the smallest processing time. In this case, this would be the merge-join if the presorted tables are available and hash-join if not combined with early or late materialization.

10. CONCLUSIONS

In this article, we presented concepts and implementations for on-the-fly hardware acceleration for almost all important operators appearing in SQL queries. Using partial dynamic reconfiguration, query-specific data paths are composed on the fly. An incoming SQL query is analyzed and divided into single operators and modules are subsequently concatenated to a data path and loaded on the FPGA. A library of partially reconfigurable modules of SQL operators has been implemented including support for new, important, and time-consuming operators such as joins and sorting. Different processing strategies, like row-wise or column-wise processing, have been analyzed and special column-wise processing modules have been introduced.

Next, a mathematical throughput analysis was presented for performance analysis of a query plan based on the provided query module library. Using this calculus, we are able to analyze the data throughput for all possible generated data paths. This is an important step to give a database management system the possibility to decide which (sub)query is worth accelerating with hardware and which is not. Finally, the throughput values of the analysis were compared with real measured values and a comparison to an x86-based software system was given.

Moreover, we proposed a PCIe-based prototype as a server extension card and SoC prototypes based on the Xilinx Zynq platform. The processing time of these prototypes was compared with an x86-based server and the partial reconfiguration overhead was shown. Finally, for a given query, different processing strategies were compared on different table sizes.

ACKNOWLEDGMENTS

The authors would like to thank Rainer Dorsch, a former member of IBM Deutschland Research & Development GmbH, for his contributions to this work.

REFERENCES

- Daniel J. Abadi, Samuel R. Madden, and Nabil Hachem. 2008. Column-stores vs. row-stores: How different are they really? In *Proc. ACM SIGMOD International Conference on Management of Data (SIGMOD'08)*, Jason Tsong-Li Wang (Ed.). ACM, 967–980.
- Andreas Becher, Florian Bauer, Daniel Ziener, and Jürgen Teich. 2014. Energy-aware SQL query acceleration through FPGA-based dynamic partial reconfiguration. In *International Conference on Field Programmable Logic and Applications (FPL'14)*. 662–669.
- Christian Beckhoff, Dirk Koch, and Jim Torresen. 2012. Go ahead: A partial reconfiguration framework. In *IEEE 20th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM'12)*. 37–44. DOI: <http://dx.doi.org/10.1109/FCCM.2012.17>
- Mark Bourgeault. 2011. Alteras Partial Reconfiguration Flow. Available online: <http://www.eecg.utoronto.ca/~jayar/FPGAseminar/FPGA-Bourgeault-June23-2011.pdf>.
- Christopher Claus, Rehan Ahmed, Florian Altenried, and Walter Stechele. 2010. Towards rapid dynamic partial reconfiguration in video-based driver assistance systems. In *Reconfigurable Computing: Architectures, Tools and Applications*. Springer, 55–67.
- Transaction Processing Performance Council. 2012. *TPC BENCHMARK™DS*. Standard Specification Version 1.1.0. TPC, www.tpc.org.
- Christopher Dennl, Daniel Ziener, and Jürgen Teich. 2012. On-the-fly composition of FPGA-based SQL query accelerators using a partially reconfigurable module library. In *IEEE 20th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM'12)*. 45–52. DOI: <http://dx.doi.org/10.1109/FCCM.2012.18>
- Christopher Dennl, Daniel Ziener, and Jürgen Teich. 2013. Acceleration of SQL restrictions and aggregations through FPGA-based dynamic partial reconfiguration. In *Proceedings of the IEEE International Field-Programmable Custom Computing Machines Symposium (FCCM'13)*. 25–28.
- John F. Gantz, Christopher Chute, Alex Manfrediz, Stephen Minton, David Reinsel, Wolfgang Schlichting, and Anna Toncheva. 2008. The Diverse and Exploding Digital Universe: An Updated

- Forecast of Worldwide Information Growth Through 2011. (2008). <http://mac3.org/t/the-diverse-and-exploding-digital-universe-e8482-pdf.pdf> An IDC White Paper – sponsored by EMC Univers.
- Robert J. Halstead, Bharat Sukhwani, Hong Min, Mathew Thoennes, Parijat Dube, Sameh Asaad, and Balakrishna Iyer. 2013. Accelerating join operation for relational databases with FPGAs. In *IEEE 21st Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM'13)*. IEEE, 17–20.
- IBM. 2014. IBM Netezza Data Warehouse Appliances. <http://www-01.ibm.com/software/data/netezza/>. (March 2014).
- William H. Inmon. 2002. *Building the Data Warehouse*. J. Wiley.
- Dirk Koch and Jim Torresen. 2011. FPGASort: A high performance sorting architecture exploiting run-time reconfiguration on FPGAs for large problem sorting. In *Proceedings of the 19th ACM/SIGDA International Symposium on Field Programmable Gate Arrays*. ACM, 45–54.
- Jonathan G. Koomey. 2008. Worldwide electricity used in data centers. *Environmental Research Letters* 3, 3 (2008), 034008.
- Peter A. Milder, James C. Hoe, and Markus Püschel. 2009. Automatic generation of streaming datapaths for arbitrary fixed permutations. In *Proceedings of the Conference on Design, Automation and Test in Europe (DATE'09)*. European Design and Automation Association, 1118–1123. <http://dl.acm.org/citation.cfm?id=1874620.1874891>
- Mark P. Mills. 2013. The Cloud Begins With Coal – Big Data, Big Networks, Big Infrastructure, and Big Power. (2013). http://www.tech-pundit.com/wp-content/uploads/2013/07/Cloud_Begins_With_Coal.pdf.
- Rene Mueller, Jens Teubner, and Gustavo Alonso. 2010. Glacier: A query-to-hardware compiler. In *Proceedings of the 2010 International Conference on Management of Data (SIGMOD'10)*. ACM, New York, NY, 1159–1162. DOI: <http://dx.doi.org/10.1145/1807167.1807307>
- SQL 2008. Information Technology Database Languages SQL Part 1: Framework (SQL/Framework). ANSI/ISO/IEC 9075-1:2008. (2008).
- Mike Stonebraker, Daniel J. Abadi, Adam Batkin, Xuedong Chen, Mitch Cherniack, Miguel Ferreira, Edmond Lau, Amerson Lin, Sam Madden, Elizabeth O'Neil, Pat O'Neil, Alex Rasin, Nga Tran, and Stan Zdonik. 2005. C-store: A column-oriented DBMS. In *Proceedings of the 31st International Conference on Very Large Data Bases*. VLDB Endowment, 553–564.
- Bharat Sukhwani, Mathew Thoennes, Hong Min, Parijat Dube, Bernard Brezzo, Sameh Asaad, and Donna Dillenberger. 2015. A hardware/software approach for database query acceleration with FPGAs. *International Journal of Parallel Programming* 43, 6 (2015), 1129–1159. DOI: <http://dx.doi.org/10.1007/s10766-014-0327-4>
- Takashi Takenaka, Masamichi Takagi, and Hiroaki Inoue. 2012. A scalable complex event processing framework for combination of SQL-based continuous queries and C/C++ functions. In *22nd International Conference on Field Programmable Logic and Applications (FPL'12)*. 237–242. DOI: <http://dx.doi.org/10.1109/FPL.2012.6339187>
- Jens Teubner, Louis Woods, and Chongling Nie. 2013. XLynx – An FPGA-based XML filter for hybrid XQuery processing. *ACM Transactions on Database Systems* 38, 4, Article 23 (Dec. 2013), 39 pages. DOI: <http://dx.doi.org/10.1145/2536800>
- Stefan Werner, Sven Groppe, Volker Linnemann, and Thilo Pionteck. 2013. Hardware-accelerated join processing in large semantic web databases with FPGAs. In *International Conference on High Performance Computing and Simulation (HPCS'13)*. IEEE, 131–138.
- Louis Woods, Gustavo Alonso, and Jens Teubner. 2013. Parallel computation of skyline queries. In *IEEE 21st Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM'13)*. IEEE, 1–8.
- Louis Woods, Zsolt István, and Gustavo Alonso. 2014. Ibex – An intelligent storage engine with support for advanced SQL off-loading. *PVLDB* 7, 11 (2014), 963–974. Proceedings of the 40th International Conference on VLDB (Hangzhou, China, Sept. 1-5).
- Xilinx. 2011. *UG743: Overview of the Partial Reconfiguration Flow*. Technical Report. Xilinx.

Received May 2015; revised September 2015; accepted November 2015